## Data Sync: Frontend ↔ Backend ↔ AI

Diese Seite beschreibt welche Daten zwischen Frontend (Rezeptsystem), Backend und AI model (bzw. trainings code) ausgetauscht werden müssen und wo diese liegen sollten.

Jeder Defekt bekommt eine eigene Klasse. Diese hat eine methode calc_features welche definiert was für diesen Defekt alles berechnet werden soll. Die Funktion schreibt alle berechnungen zurück in DefectsNew und ändert ggf die Klasse des Defekts. Achtung: In der DefectsNew klasse muss es für alle informationen die es in mindestens einem defekt gibt eine spalte für alle defekte geben, ob das sinn macht oder nicht (wegen Vektorisierung).

```
DefectBase(abc, dataclass):
  name: str = "BaseDefect" # name of this defect
  subtypes: List[str] = ["subtype1", "subtype2"] # subtypes if any
  output_mapping: Dict = {"length": ("min_encl_rect_lengths_mm", "mm"),
                          "width": ("min_encl_rect_widths_mm", "mm"),
                          "aspect_ratio": ("min_encl_rect_asp_ratios", N
  generate_schema() -> json
  calc_features(logger: LogCollector, images: List[np.array],
                defects: DefectsNew, defect_idx: int, px_size_um: int,
                full_img_shape: Tuple[int, int], angle_ref_points: List
                sheet_points: np.array, roi: np.array) → DefectsNew
```

Beispiel Defekt (Knot), muss für jeden Kunden neu angelegt werden. JB → Jogeva Buche

```
KnotJB(DefectBase):
  name: str = "Knot"
  subtypes: List[str] = ["Sound Knot", "Dark Knot", "Sound Knot with Ho
                          "Dark Knot with Hole"]
  output_mapping: Dict = {"length": ("min_encl_rect_lengths_mm", "mm"),
                          "width": ("min_encl_rect_widths_mm", "mm"),
                          "aspect_ratio": ("min_encl_rect_asp_ratios", N
                          "area": ("contours_area_mm2", "mm2")}
  generate_schema() -> json
  calc_features(logger: LogCollector, images: List[np.array],
                defects: DefectsNew, defect_idx: int, px_size_um: int,
                full_img_shape: Tuple[int, int], angle_ref_points: List
                sheet_points: np.array, roi: np.array) → DefectsNew
```

#### Customer (Enum) [str]

JOGEVA = “jogeva”
DUKE = “duke”

#### BuildingMaterial (Enum) [str]

BEECH = “beech”
GYPSUM_BOARD = “gypsum_board”
...

#### DefectsBase (abc, dataclass)

defects: List[DefectBase]
generate_schema() → json (generates json code with all defect information to satisfy pilzmasters frontend)

#### DefectsJogevaBeech1 (DefectsBase )

defects: List[DefectBase]= [KnotJB, BarkJB, OpeningJB, CambiumJB, DiscolorationJB, StripeJB, DecayJB, WatermarksJB, CalmnessJB, HarvestermarksJB, KnifemarksJB, Colored LinesJB, RoughnessJB, WavinessJB, FragmentJB, OverlappingJB, TapeJB]

#### ActiveDefects (Enum)

JOGEVA_BEECH_1: DefectsJogevaBeech1
DUKE_GYPSUM_BOARD_1:

#### Defekt subtype Mapping

Sound Knot: Knot
Dark Knot: Knot
Sound Knot + Hole: Knot
Dark Knot + Hole: Knot
Bark + Hole: Bark
Hole: Opening
Split: Opening
Micro Split: Opening
...

### Plugin System

## Architektur-Konzept: Dynamische KI-Modelle & OpenCV-Plugins

### 🎯 Ziel

Das Frontend (Svelte) soll dynamisch Rezepte basierend auf dem aktuell gewählten Modell erstellen können. Das Backend (Python/PyTorch/OpenCV) muss austauschbare Modelle und modellspezifische Nachverarbeitungs-Regeln ausführen können, **ohne** dass der Kern-Code des Backends bei neuen Modellen aktualisiert werden muss.

### 📦 1. Das Austauschformat: `.aiaddon`

Ein neues Modell wird als ZIP-Archiv (mit der Endung `.aiaddon`) ausgeliefert.

**Struktur des Archivs:**

- `manifest.json` (Klartext) → *Sagt dem System, was das Modell kann.*
- `model.pth.enc` (Verschlüsselt) → *Die PyTorch-Gewichte.*
- `plugin.py.enc` (Verschlüsselt) → *Spezifische OpenCV-Regeln für dieses Modell.*

**Warum so?** Das Svelte-Frontend kann die `manifest.json` lokal im Browser lesen und validieren (z. B. Modellnamen anzeigen, Prüfsummen checken), ohne jemals den geheimen Entschlüsselungs-Key zu benötigen.

---

### 📄 2. Die `manifest.json` (Single Source of Truth)

Diese Datei wird vom Svelte-Frontend genutzt, um die UI dynamisch aufzubauen, und vom Backend, um zu wissen, welche Plugins geladen werden müssen.

```jsonc
{
  "model_id": "beech_v2",
  "ai_classes": {
    "knot": 6,       // KI findet Klasse 6, wir nennen es "knot"
    "bark": 0
  },
  "recipe_schema": {
    // Hier steht exakt Ihr bisheriges Frontend-JSON.
    // Svelte liest das und baut Dropdowns/Slider.
  },
  "post_processing_rules": [
    {
      "id": "knot_intensity_split",
      "applies_to_ai_class": "knot",
      "plugin_class": "KnotIntensityRule",
      "outputs_subclasses": ["knot_dark", "knot_sound"]
    }
  ]
}

```

---

### 🔄 3. Anpassungen im Python-Backend

#### A. Dynamische Datenerfassung (Die `DefectsNew`-Klasse)

**Was bleibt:** Die SoA (Structure of Arrays) Architektur für Standard-Geometrie (Bounding Boxes, Area, etc.) bleibt unangetastet für maximale Performance. **Was sich ändert:** Wir fügen Container für die dynamischen Plugin-Ergebnisse hinzu.

```python
@dataclass
class DefectsNew(Result):
    # --- BLEIBT WIE BISHER ---
```

```
    base_ai_classes: I32Array = empty_i32() # z.B. 6 (Ast)
    corners: F32Array = empty_corners_2()
    contours_area_px: I32Array = empty_i32()
    # ...

    # --- NEU HINZUFÜGEN ---
    # Was das Plugin daraus macht (z.B. "Ast_hell")
    derived_classes: I32Array = empty_i32()

    # Dictionary für dynamische Arrays aus den Plugins
    # Key: "intensity_score", Value: np.array([120.5, 0.0, 200.1, ...])
    custom_numeric_features: Dict[str, np.ndarray] = field(default_fact

```

#### B. Die Kern-Schleife ( `extract_features` )

**Was bleibt:** Das Extrahieren der Bilder und Berechnen der Standard-Features. **Was sich ändert:** Am Ende der Funktion rufen wir dynamisch das Plugin auf, falls das Frontend-Rezept dies verlangt.

```
def extract_features(defects, recipe_from_frontend, ...):

    # 1. Standard-Features berechnen (Ihr bestehender Code!)
    calculate_standard_geometry(defects)

    # 2. NEU: Plugins für spezifische OpenCV-Regeln ausführen
    plugin = load_plugin_for_current_model()

    for rule in recipe_from_frontend.active_rules:
        # Führt z.B. den "knot_intensity_split" aus
        plugin.execute_rule(
            rule_name=rule.id,
            defects=defects,
            params=rule.params
        )

    return JobResult(result=defects)

```

#### C. Das Modell-Plugin ( `plugin.py` )

Dieser Code wird **mit dem Modell ausgeliefert** und vom Backend dynamisch geladen. Er füllt die neuen dynamischen Felder aus.

```
# plugin.py (Verschlüsselt im .aiaddon)
import numpy as np

class KnotIntensityRule:
    def execute(self, defects, params, images):
        threshold = params.get("threshold", 128)

        # Leeres Array für das neue Feature anlegen
        if "intensity_score" not in defects.custom_numeric_features:
            defects.custom_numeric_features["intensity_score"] = np.zer

        # Nur Äste (Klasse 6) verarbeiten
        knot_indices = np.where(defects.base_ai_classes == 6)[^0]

        for idx in knot_indices:
            # OpenCV Logik...
            mean_intensity = calculate_mean(images, defects.corners[idx
```

```

            # Ergebnis in dynamisches Array schreiben
            defects.custom_numeric_features["intensity_score"][idx] = m

            # Subklasse definieren
            defects.derived_classes[idx] = 61 if mean_intensity > thres

```

---

### 🌐 4. Workflow (Zusammenfassung)

1. **Upload:** Der User wählt in Svelte die Datei `beech_v2.aiaddon`. Svelte liest die unverschlüsselte `manifest.json`, sagt "Valides Modell erkannt" und lädt sie ins Backend hoch.
2. **Install:** Das Backend entschlüsselt die `.pth` und `.py` Dateien mit seinem geheimen Key und speichert sie ab.
3. **UI-Sync:** Svelte fragt `/api/models/beech_v2/schema` ab. Das Backend sendet die JSON. Svelte baut daraus den Rezept-Editor (User kann nun "Ast dunkel" auswählen).
4. **Inferenz:** Die KI findet einen Defekt (Klasse 6). Das Backend berechnet die Fläche. Das Backend schaut ins Rezept, sieht die Helligkeits-Regel, ruft die `plugin.py` auf, welche den Ast in "Ast dunkel" klassifiziert und den Score im Dictionary `custom_numeric_features` speichert.
5. **Ergebnis:** Das Frontend erhält den finalen Defekt als JSON und kann ihn anzeigen.